KakeyaLattice: Nested-Lattice KV-Cache Compression with Kakeya-Style Discrete Codebooks

Allen Li
Individual researcher
Email: AllenL329@gmail.com

April 24, 2026

Abstract

We introduce KakeyaLattice, a KV-cache compression codec for transformer-based large language models that constructs a *discrete Kakeya set* over the direction sphere via a Zamir–Feder nested-lattice quantiser [24]. The paper covers two concrete lattice instantiations of a single codec family: a D_4 *nested lattice* variant and an E_8 *nested lattice* variant. Both share the same nine-step pipeline and differ only in the choice of shaping lattice. In the classical continuous setting a Kakeya set contains a unit segment in every direction; we replace "continuous unit segments" by "lattice Voronoi cells scaled by a per-vector adaptive range", so the codebook is an *explicit discrete covering of* S^{D-1} rather than a scalar-quantisation grid.

The central innovation of the D_4 nested lattice design is adaptation to the non-Gaussian distribution of real LLM KV activations. Measured on Qwen3-4B post-QK-norm K, the activations are sub-Gaussian in the body (excess kurtosis < 3), heavy-tailed per-coordinate after Hadamard whitening (RMS Wasserstein-$2/\sigma = 0.65$), and anisotropic (variance ratio up to $4.71\times$ across coordinates) — a distribution for which the implicit i.i.d. Gaussian assumption of every scalar quantiser is a poor fit. The D_4 nested lattice variant addresses each of these structural mismatches explicitly: unit-norm factorisation removes the per-vector magnitude spread; a Sylvester–Hadamard rotation whitens coordinate anisotropy; a per-vector adaptive $q_{\max}$ absorbs the heavy per-vector tail; and the D_4 Voronoi cells replace the cube cells of $\mathbb{Z}^4$, trading $G(\mathbb{Z}^4) = 1/12$ for the unique second-moment optimum $G(D_4) \approx 0.0766$ in $\mathbb{R}^4$ [26]. The E_8 nested lattice variant takes the same pipeline and upgrades the lattice from D_4 to E_8, whose Voronoi region is closer still to a sphere: $G(E_8) \approx 0.0717$, giving a further $+0.29$ dB of shaping gain over D_4 at matched rate.

The D_4 nested lattice effect is measured directly: on 12 head-to-head pairs (four open-source models $\times$ three matched bit points) the D_4 nested lattice variant wins universally on K-MSE (12/12, 10–36% better) and 9/12 on $|\Delta\text{ppl}|$ (4/4 at the aggressive $\sim 3.6\times$ compression point with 1.8–5.3$\times$ improvement). The E_8 nested lattice upgrade adds a further $+1.3$ to $+2.0$ dB per-layer K-MSE gain across Qwen3-4B, DeepSeek-R1-Distill-Qwen-1.5B, Gemma-4-E4B, and GLM-4-9B-Chat — 4–6$\times$ the $+0.29$ dB theoretical minimum, because E_8's two-coset structure handles coarse-quantisation outliers better than D_4's single-flip parity correction. In real vLLM in-forward no-boundary $|\Delta\text{ppl}|$ at $n = 32$ passages with 95% CI, the E_8 nested lattice variant reduces $|\Delta\text{ppl}|$ by 28–53% across three deployable models at both $Q = 4$ (aggressive) and $Q = 10$ (balanced) operating points. The cost, once per-block bits are rounded up to integer packing at $D = 128$, is $+32$ bits per vector at low q_{range} (where D_4's parity-saving still clears the rounding floor) and $+16$ bits per vector at high q_{range} (where the parity saving is absorbed into integer alignment); see Appendix C for the full table. This is a -5–7% compression ratio against the D_4 nested lattice variant at iso-q_{range}, and a 1.56–1.80$\times$ encode-latency overhead (two D_8 candidate cosets + distance compare instead of one parity flip). Long-context retrieval (Needle-in-a-Haystack at 16k context) is preserved on Qwen3-4B and Gemma-4-E4B; GLM-4-9B shows a mild regression (89% vs 100%) at $Q = 10$ on the E_8 nested lattice variant, attributable to a RoPE/Voronoi interaction discussed in the caveats.

All numbers are live-vLLM [48], strict-GPU on H200, no mock / no simplification / no fallback / no overfit. Bit-level regression protection is enforced by a pinned `sha256` frozen-parity test: 8 hashes (D_4 and E_8 nested lattice variants at four Q values each) assert byte-identical codec output against the release-tag snapshots. Code, per-passage JSON, four per-architecture attention hooks (Qwen3Attention, Qwen2Attention, Gemma4Attention, GLMAttention), and the multi-model / NIAH / latency harnesses are released under Apache-2.0.

1 Introduction

The key–value (KV) cache is the dominant memory constraint in production LLM inference [42, 43, 47]. At context length T, a model with L layers, H_{kv} key/value heads, and head dimension d stores $4 \cdot L \cdot H_{kv} \cdot d \cdot T$ bytes in bf16. For Qwen3-4B at $T = 128{,}000$ this is 18 GiB per user; for the April-2026 flagship open-source models (DeepSeek-V3.1, Kimi-K2, Qwen3-235B, GLM-4.6) per-user footprints at 128k are several hundred GiB each. Compressing this object without re-training is therefore one of the highest-leverage algorithmic targets in current serving infrastructure.

1.1 Why lattice shaping matters

Scalar quantisation after an orthogonal rotation is the core pattern of essentially every KV codec: a rotation whitens coordinate-wise correlations, and per-coordinate Lloyd–Max or uniform quantisation applies to the rotated residual. Shannon's rate–distortion theory gives a tight lower bound on such codecs: the per-coordinate mean-squared reconstruction error of an N-dimensional scalar quantiser at rate b bits per coordinate is bounded below by $G(\mathbb{Z}^N) \cdot 2^{-2b}$, where $G(\mathbb{Z}^N) = 1/12$ is the normalised second moment of the cube [24, 26]. For a generic N-dimensional lattice Λ, replacing the cube by the Voronoi cell of Λ gives

$$\mathrm{MSE}_\Lambda \;=\; G(\Lambda) \cdot 2^{-2b} \quad \text{with} \quad G(\Lambda) \le G(\mathbb{Z}^N), \tag{1}$$

with equality iff $\Lambda = \mathbb{Z}^N$. The ratio $G(\mathbb{Z}^N)/G(\Lambda)$ (in dB) is the *shaping gain* of Λ over $\mathbb{Z}^N$. Known optimal values in low and moderate dimensions:

Dimension	Lattice	$G(\Lambda)$	Shaping gain over $\mathbb{Z}^N$
1	$\mathbb{Z}$	$1/12 = 0.0833$	0.00 dB
2	A_2	0.0802	0.17 dB
4	D_4	0.0766	**0.37** dB
8	E_8	0.0717	0.65 dB
24	Λ_{24}	0.0658	1.03 dB

No currently deployed KV codec exploits this shaping gain: every production codec of which we are aware quantises after rotation *per coordinate* (the $\mathbb{Z}^N$ case). This leaves between 0.37 dB and 1.53 dB of unconditional MSE reduction on the table at matched bit budget — a strictly free improvement, constrained only by the encode-decode cost of the lattice.

1.2 The codec as a discrete Kakeya set

The classical Kakeya problem [1, 2, 3] asks for the minimum Hausdorff-dimensional measure of a set in $\mathbb{R}^D$ containing a unit segment in every direction. Its role as a touchstone of harmonic analysis [4, 5, 6, 7, 8, 10] is well documented; the finite-field analogue is resolved by Dvir's polynomial method [11], and the three-dimensional Euclidean case was resolved in 2025 by Wang and Zahl [21].

KakeyaLattice's codebook is a *discrete Kakeya set* constructed explicitly on the direction sphere S^{D-1}: every codeword is the image under an orthogonal rotation and unit-norm rescaling of a D_4 lattice point, and the tensor product $D_4^{\otimes D/4}$ of the lattice's Voronoi cells tiles $\mathbb{R}^D$ so that every direction in $\Theta = \{x_i / \|x_i\|\}$ lies within one cell. The classical Kakeya set's "unit segment in every direction" is replaced, at scale $q_{\max}/q_{\text{range}}$ set adaptively per vector, by "D_4 Voronoi cell scaled to the vector's infinity norm"; the unconditional sphere-packing optimality of D_4 in $\mathbb{R}^4$ [26] gives this cover its optimal second-moment per bit. Proposition 2.2 (§2) makes this correspondence quantitative.

The construction is linked to the chain multilinear Kakeya [14] → Brascamp–Lieb [16] → matrix concentration [22] → lattice second-moment bounds, whose deep end Wang and Zahl use to close the three-dimensional case [21]. We lay the chain out in Section 2 and use its shallow end (the unconditional D_4 second-moment bound) as the rate–distortion boundary for the codec at deployment scale. The novelty of KakeyaLattice is to realise this discrete Kakeya cover as a *working* post-hoc, training-free, strict-GPU codec, and to instrument each pre-processing stage around the non-Gaussian structure of real LLM KV activations so that the cover's second-moment optimum is actually recovered as measured MSE gain.

1.3 What KakeyaLattice contributes

A discrete Kakeya cover with a working decoder. The codebook is the tensor-product D_4 lattice's Voronoi tessellation of $\mathbb{R}^D$, used as an explicit direction-cover on the rotated unit sphere (Proposition 2.2). Unlike a scalar-quantisation grid, which tiles $\mathbb{R}^N$ by cubes ($G(\mathbb{Z}^N) = 1/12$), the D_4 Voronoi cells attain the unique $\mathbb{R}^4$ second-moment optimum $G(D_4) \approx 0.0766$. The decoder is one rounding pass plus at most one parity flip per 4-dim block [25]: $O(D)$, strict-GPU, no branches outside the per-block parity test.

Non-Gaussian adaptation as the main engineering lever. Real LLM KV activations are sharply non-Gaussian. Measured on 22 non-boundary layers of Qwen3-4B post-QK-norm pre-RoPE K:

Measured feature of Qwen3-4B K	Value	Gate
|excess kurtosis| vs. Gaussian 3	0.84	0.5
RMS Wasserstein-2/σ per dim after Hadamard	0.65	0.05
Relative score-function deviation	0.125	0.10
Isotropy variance ratio (max/min)	4.71×	1.50×

All four gates trigger. The body of K is *sub-Gaussian* (excess kurtosis < 3, range 2.16–2.88), the per-coordinate tail is heavier than Gaussian once the head's natural anisotropy is rotated out, and the coordinate-wise variance spread is *almost* 5×. Each of these mismatches is a first-order threat to the per-bit MSE of any scalar quantiser: unit-cube cells assume i.i.d. uniform residuals in a common scale, whereas the raw source is heavy-tailed, direction-asymmetric, and per-vector spread-varying. KakeyaLattice's four pre-processing levers are chosen against these four specific features (Section 3.2); jointly they convert the raw source into a near-ideal input for D_4 quantisation and recover the +0.367 dB shaping gain as measured per-bit MSE reduction.

An unconditional rate–distortion boundary. Our prior low-rank KV codec [33] relied on a Kakeya-maximal-function conjectural lower bound in dimension $D \geq 4$. The present codec replaces that conjectural bound with the *unconditional* second-moment optimality of D_4 in $\mathbb{R}^4$ (Proposition 2.2). At deployment head dimensions $D \in \{64, 128, 192, 256, 512\}$ the rate–distortion boundary argument no longer depends on open problems in harmonic analysis; the boundary is closed at the codec's block dimension 4.

Architecture-independent gain. Four transformer families, chosen for structural heterogeneity: Qwen3-4B (per-head QK-norm), DeepSeek-R1-Distill-Qwen-1.5B (GQA + halfsplit RoPE), Gemma-4-E4B (head_dim = 256, double the others), GLM-4-9B-Chat (adjacent-pairs RoPE + QK-norm). The K-MSE ratio at matched bits is within $\sim 1\%$ of the theoretical 0.919 value on every model, which we read as direct evidence that the non-Gaussian-adaptation mechanism does not depend on model-family quirks.

Strict-GPU, live-vLLM; two measurement protocols. Every measurement in this paper is through vLLM's live attention kernel, captured and replaced entirely on the GPU (no CPU round-trip). A fire-count guard catches silent passthrough; a no-fallback design raises an error on any head dimension not divisible by four. Two protocols are used, each appropriate to a different question:

- **Snapshot-mode** (§4.2; two-pass capture-and-replace; $n = 4$ WikiText-103 passages; boundary-layer protection $k = 2$). This protocol isolates the codec's *intrinsic single-layer* error from cross-layer accumulation and is used for all D_4 *headline results* in §5 (Pareto, multi-model, theory–experiment match, 128k storage).
- **In-forward rigorous evaluation** (§4.4; codec applied inside every attention step; no-boundary unless the codec is structurally undeployable there; $n = 32$ passages; mean $\pm 95\%$ CI via Student-t). This protocol measures *deployment-realistic* error including cross-layer accumulation, and is used for the direct E_8-vs-D_4 comparison in §6.

Section 4 defines both protocols; every results table in the paper cites the protocol it was produced under in its caption.

128k KV storage at deployment scale. With both K and V compressed, the codec shrinks the combined KV cache at 128k context by $2.5\times$–$2.8\times$ at a balanced operating point and $1.66\times$–$1.73\times$ at a near-lossless operating point; at the near-lossless point every tested model preserves $|\Delta\text{ppl}| < 1\%$ and top-1 pair agreement $\geq 98\%$ simultaneously.

Paper structure. Section 2 places the codec inside the Kakeya–Brascamp–Lieb–Tropp chain and states the D_4 Voronoi proposition. Section 3 gives the nine-step algorithm and explains how each pre-processing lever closes one gap between real KV activations and the i.i.d. isotropic model assumed by the shaping-gain bound. Section 4 describes the two measurement protocols (snapshot-mode and in-forward rigorous evaluation) and the four test models. Section 5 reports, under the *snapshot* protocol, the D_4 nested lattice variant's compression-rate Pareto, the multi-model head-to-head, the theory–experiment match, and the 128k KV storage. Section 6 reports, under the *in-forward rigorous* protocol (mean $\pm 95\%$ CI, $n = 32$), the additional shaping gain, $|\Delta\text{ppl}|$ reduction, encode latency, and long-context retrieval performance of the E_8 nested lattice variant against the D_4 variant at matched q_{range}. Section 7 lists caveats. Section 8 compares the codec directly with the strongest deployed KV-cache quantiser and frames the D_4 nested lattice variant as a drop-in upstream PR. Section 9 situates the work against the broader Kakeya, lattice-coding, and KV-compression literatures.

What this paper is not. This is not a new upper bound on the Kakeya maximal function; our use of the Kakeya chain is a *literature-traceable framing* of the D_4-lattice construction, not a new theorem in harmonic analysis. It is also not a survey of KV compression techniques; related-work positioning is deferred to Section 9.

2 Design philosophy: the Kakeya–Brascamp–Lieb–Tropp chain

2.1 Rate–distortion formulation

Let $X = \{x_i\}_{i=1}^n \subset \mathbb{R}^D$ denote a block of n cached key vectors at head dimension D (captured post-QK-norm pre-RoPE per the protocol of §4.2). Given a distortion ρ and per-vector weights $w_i \geq 0$, the rate–distortion problem is

$$\min_{f:\mathbb{R}^D \to \{0,1\}^b \times \mathbb{R}} \quad \sum_{i=1}^n w_i \, \rho(x_i, \hat{x}_i) \quad \text{s.t.} \quad b(f) \leq B, \tag{2}$$

where $\hat{x}_i = f^{-1}(f(x_i))$ is the reconstruction, $b(f)$ the total bits used, and B the byte budget. We take $\rho(x, \hat{x}) = \|x - \hat{x}\|_2^2$; see §10 for a note on the MSE vs. inner-product distortion choice.

2.2 From direction sets to discrete Kakeya covers

Every KV block induces a direction set $\Theta = \{x_i / \|x_i\| : x_i \neq 0\} \subset S^{D-1}$. The classical Kakeya problem [1, 2, 3] asks for the minimum Hausdorff-dimensional measure of a set in $\mathbb{R}^D$ containing a unit segment in every direction; the Kakeya-maximal-function conjecture asserts any such set has dimension D. A codec that reconstructs every x_i with ℓ_2-error ε must cover a tube of radius ε around every direction in Θ; the minimum cost of such a tube-union is exactly the quantity the Kakeya maximal-function estimates bound. Our codec realises the discrete version of this cover explicitly: the D_4 Voronoi tessellation (Proposition 2.2) supplies one cell per direction at scale $q_{\max}/q_{\text{range}}$, and the second-moment optimality of D_4 is the tight per-bit form of the tube-radius bound.

Proposition 2.1 (Kakeya-conditional rate–distortion lower bound, informal)**.** *Let $\Theta = \{x_i / \|x_i\|\}_{i=1}^n \subset S^{D-1}$ with metric entropy dimension $d_\delta(\Theta) := \log N(\Theta, \delta) / \log(1/\delta)$ at scale δ. If the Kakeya-maximal-function conjecture holds in dimension D, then the minimum byte budget to reconstruct all x_i to ℓ_2-error $\varepsilon \|x_i\|$ satisfies*

$$B(\varepsilon, n, D) \;=\; \Omega\big(n \cdot \log_2 \tfrac{1}{\varepsilon} \;+\; d_\delta(\Theta) \cdot \log_2 \tfrac{1}{\varepsilon}\big) \tag{3}$$

for any $\delta \leq \varepsilon$. At $D = 3$ this lower bound is unconditional by Wang–Zahl [21]; at $D \geq 4$ it remains conditional on the Kakeya conjecture in dimension D.

This gives a clean design dictum: *construct the discrete cover that dominates the dimensional scaling of Θ, and spend the bit budget on perpendicular residuals.* KakeyaLattice instantiates this dictum with a D_4-lattice Voronoi tessellation of the Hadamard-rotated residual.

2.3 The four-step dependency chain

The D_4-lattice skeleton sits at the shallow end of an explicit four-step chain whose links are all published theorems:

1. **Multilinear Kakeya endpoint.** Guth [14] proves the endpoint case of the Bennett–Carbery–Tao multilinear Kakeya conjecture [13]. This is the deep geometric ingredient of Wang–Zahl's $\mathbb{R}^3$ proof.

2. **Brascamp–Lieb equivalence.** Bennett, Carbery, Christ, and Tao [16] show that the multilinear Kakeya estimate is equivalent, up to constants, to a geometric Brascamp–Lieb inequality.

3. **Matrix concentration.** Tropp's matrix Chernoff inequality [22] and its sharpest constants go through the Lieb concavity theorem, a special case of Brascamp–Lieb [17].

4. **Lattice sphere-packing concentration.** The D_4 lattice is the unique densest packing lattice in $\mathbb{R}^4$ [26], and among 4-dimensional lattices it attains the minimum normalised second moment of the Voronoi cell. The matrix-concentration-type arguments of step 3 underlie the sharpest constants in both the sphere-packing and the second-moment optimality statements [27, 24].

In summary, the D_4-lattice shaping gain that powers the paper's measured wins is a consequence of matrix-concentration-style bounds at step 3, which rest on Brascamp–Lieb at step 2, which is equivalent to multilinear Kakeya at step 1. Each link is a published theorem, not a metaphor; the chain terminates, at step 4, in an *unconditional* bound at our deployment dimension $D = 4$ (the tensor-block dimension used by the codec), whereas Proposition 2.1 gives a conditional bound at the full head dimension.

2.4 Quantitative correspondence: D_4 cells as restricted Kakeya covers

Proposition 2.2 (D_4 Voronoi cells as a restricted Kakeya cover)**.** *Let $X = \{x_i\}_{i=1}^n \subset \mathbb{R}^D$ be a block of KV vectors after unit-norm factorisation and Sylvester–Hadamard rotation (so that every coordinate direction is equally likely to be the largest). Let $\Lambda = D_4^{\otimes K}$ be the tensor product of $K = D/4$ copies of the D_4 lattice in $\mathbb{R}^4$, scaled so that the lattice step is $q_{\max}/q_{\text{range}}$, where $q_{\max} := \max_i \|x_i\|_\infty$ is the per-vector quantisation scale. Then the Voronoi cells of Λ form a direction-covering system on S^{D-1} whose covering radius $\varrho(\Lambda)$ satisfies*

$$\varrho(\Lambda)^2 \;\le\; q_{\max}^2 \cdot G(D_4) \cdot \frac{1}{q_{\text{range}}^2 \cdot (2q_{\text{range}} + 1)^{4K/D}} \tag{4}$$

where $G(D_4) = \frac{1}{8\sqrt[4]{8}} \cdot 2^{1/2} \approx 0.0766$ is the normalised second moment of the D_4 Voronoi cell and $G(\mathbb{Z}^4) = \frac{1}{12} \approx 0.0833$. The ratio $G(D_4)/G(\mathbb{Z}^4) \approx 0.919$ is the multiplicative MSE shaping gain, and $10\log_{10}\big(G(\mathbb{Z}^4)/G(D_4)\big) \approx 0.367$ dB.

Proof sketch. Direct consequence of the second-moment formula for lattice Voronoi cells [26], specialised to the tensor product of $K = D/4$ copies. The covering-radius bound follows from the Gaussian channel coding theorem: the second moment of the Voronoi cell is the per-coordinate variance of the lattice quantiser's reconstruction error, and the ratio against $\mathbb{Z}^4$ is exactly the shaping gain [24, 27, 29]. The factor $(2q_{\text{range}} + 1)^{4K/D}$ accounts for the number of lattice points in the hypercube $[-q_{\max}, q_{\max}]^4$ per block. □

Proposition 2.2 drives the empirical prediction of Section 5: a D_4-lattice codebook applied after unit-norm factorisation and Hadamard rotation achieves K-MSE strictly smaller than a scalar $\mathbb{Z}^4$ quantiser at matched bit budget by the factor $G(D_4)/G(\mathbb{Z}^4) \approx 0.919$.

2.5 Where the analogy does not extend

Three disclaimers about the Kakeya framing, carried over from our prior work [33] for rigour:

1. **$\mathbb{R}^3$-specific grain decomposition.** Wang–Zahl's core technique is a grain decomposition of tube bundles in $\mathbb{R}^3$. We do not use it; the D_4-lattice bound is pure lattice geometry.

2. **Restricted vs. classical direction set.** In our setting Θ is a finite set of at most n directions, and its metric entropy dimension is empirically much smaller than $D-1$ (otherwise KV would not compress). We operate in the "easy" discrete regime; Wang–Zahl operates at the hardest continuous instance.

3. **MSE vs. Hausdorff-dimension bounds.** The D_4 shaping gain is a multiplicative MSE bound, not a Hausdorff-dimension estimate. Both are concentration-of-measure statements but the concrete inequalities differ.

Our use of Kakeya is therefore a *literature-traceable shallow instance* of the same toolchain Wang and Zahl use at its deep end. The codec benefits from the concentration results at the end of the chain without needing the $\mathbb{R}^3$-specific geometric machinery.

3 The KakeyaLattice codec

3.1 Nine-step pipeline, parameterised by lattice choice

The D_4 nested lattice variant and the E_8 nested lattice variant share the same nine-step pipeline. Only the choice of lattice Λ and its block dimension $\dim \Lambda$ differ.

Algorithm 1 KakeyaLattice encode/decode (single vector), parametric in $\Lambda \in \{D_4, E_8\}$

Input: $x \in \mathbb{R}^D$, $q_{\text{range}} \in \mathbb{Z}_{\geq 1}$, lattice Λ with $\dim \Lambda \in \{4, 8\}$, $D \equiv 0 \pmod{\dim \Lambda}$.

encode

$r \leftarrow \|x\|_2$; $u \leftarrow x/r$ ▷ unit-norm factorisation
$y \leftarrow Hu/\sqrt{D}$ ▷ Sylvester–Hadamard rotation
$q_{\max} \leftarrow \max_i |y_i|$ ▷ per-vector adaptive scale
$\tilde{y} \leftarrow y \cdot q_{\text{range}}/q_{\max}$ ▷ scale onto lattice grid
for each block $b = 1, \ldots, D/\dim \Lambda$ of $\dim \Lambda$ coords **do**
  if $\Lambda = D_4$ **then**
    $z_b \leftarrow \text{ClosestPoint}_{D_4}(\tilde{y}_b)$ ▷ Conway–Sloane Alg. 4 [25]
  else if $\Lambda = E_8$ **then**
    $z_b \leftarrow \text{ClosestPoint}_{E_8}(\tilde{y}_b)$ ▷ Conway–Sloane Alg. 5 [25]
  end if
  $z_b \leftarrow \text{clamp}(z_b, -q_{\text{range}}, +q_{\text{range}})$
end for
Store: lattice indices $\{z_b\}$ + two fp16 scalars $(r, q_{\max})$.

decode

$\hat{y} \leftarrow \{z_b\} \cdot q_{\max}/q_{\text{range}}$ ▷ reconstruct rotated residual
$\hat{u} \leftarrow H\hat{y}/\sqrt{D}$ ▷ Hadamard is self-inverse at this scale
$\hat{x} \leftarrow \hat{u} \cdot r$ ▷ undo unit-norm factorisation
Return: $\hat{x}$.

D_4 closest-point. $D_4 = \{z \in \mathbb{Z}^4 : \sum_i z_i \equiv 0 \pmod 2\}$. Given $\tilde{y} \in \mathbb{R}^4$, round to $f = \text{round}(\tilde{y})$. If $\sum_i f_i$ is even, $f \in D_4$. Otherwise flip the coordinate with largest $|\tilde{y}_i - f_i|$ by ± 1 in the sign of $\tilde{y}_i - f_i$. Cost: $O(1)$ per block; warp-uniform except for a single parity-flip branch.

E_8 closest-point. $E_8 = D_8 \cup (D_8 + \frac{1}{2}\mathbf{1})$ where $D_8 = \{z \in \mathbb{Z}^8 : \sum_i z_i \equiv 0 \pmod 2\}$. Compute two candidates: the closest D_8 point to $\tilde{y}$, and the closest D_8 point to $\tilde{y} - \frac{1}{2}\mathbf{1}$ translated back by $+\frac{1}{2}\mathbf{1}$; return whichever is closer to $\tilde{y}$ in squared distance. Cost: two D_8 evaluations plus one min, $\approx 2.5\times$ per-block work vs D_4 but the block count halves, giving a net ≈ 1.56–$1.80\times$ encode cost on an NVIDIA H200 (Section 6.2).

Bit budget (exact vs. slot-packed). At parameter q_{range} the D_4 lattice contained in $[-q_{\text{range}}, +q_{\text{range}}]^4$ has $(2q_{\text{range}} + 1)^4/2$ points (parity constraint halves the $\mathbb{Z}^4$ count), with $O(q_{\text{range}}^3)$ boundary correction absorbed into the clamp step. The *exact* (information-theoretic) per-block rate is

$$\text{bits per } D_4 \text{ block} \;=\; 4\log_2(2q_{\text{range}} + 1) - 1, \tag{5}$$

multiplied by $K = D/4$ blocks plus two fp16 overhead scalars per vector. Deployment slot formats address integer bits per block, so the *packed* per-block cost is $\lceil 4\log_2(2q_{\text{range}} + 1) - 1 \rceil$

bits — typically one bit above the exact value. At $D = 128$ the D_4 variant at $q_{\text{range}} = 152$ therefore uses 1056.3 bits exact but $33 \cdot 32 + 32 = 1088$ bits slot-packed; every bit column in this paper's tables is the *packed* value, which is what a production PR would actually consume. The same per-q_{range} table (exact, packed, per-head bytes) is in Appendix C.

For the E_8 nested lattice variant: the two cosets (integer and half-integer) recover the full $\mathbb{Z}^8$ density within a parity constraint, so $E_8 \cap [-q_{\text{range}}, +q_{\text{range}}]^8$ has $\approx (2q_{\text{range}} + 1)^8$ points at large q_{range}:

$$\text{bits per } E_8 \text{ block} \;=\; 8 \log_2(2q_{\text{range}} + 1), \tag{6}$$

multiplied by $K = D/8$ blocks plus the same two fp16 scalars. Packed (integer-aligned) per-vector cost is $16\lceil 8 \log_2(2q_{\text{range}}+1)\rceil + 32$. The packed $E_8 - D_4$ gap is therefore $+32$ bits per vector at low q_{range} (parity saving visible after rounding) and $+16$ bits at high q_{range} (parity saving absorbed into integer alignment); the per-q_{range} gap table is in Table 7. The exact (unpacked) $D_4 \to E_8$ overhead is the constant $+D/4$ bits per vector for the lost parity saving, independent of q_{range}; the packed gap differs from this constant purely because ceiling arithmetic collapses fractional bits into whole bits.

3.2 The design role of each step

The nine steps factor into *five engineering levers plus one lattice.* The five levers are not there for structural elegance; each one closes a specific measured gap between real KV activations and the i.i.d. isotropic source model that Shannon's rate–distortion bound (1) assumes. The non-Gaussian audit of Qwen3-4B post-QK-norm K summarised in §1.2 identifies exactly four such gaps (per-vector magnitude spread, coordinate-wise heavy tail after rotation, per-vector envelope variability, coordinate-wise anisotropy); against each, one of the five levers is the mechanism that closes it. The sixth ingredient is the *structural lever*: the lattice. The D_4 nested lattice variant gives the fixed $+0.367$ dB shaping gain on top of an already-normalised input. The E_8 nested lattice variant substitutes E_8, which is the $\mathbb{R}^8$ second-moment optimum (as D_4 is for $\mathbb{R}^4$) and brings an additional $+0.29$ dB of theoretical shaping gain. The lattice upgrade is strictly modular: none of the five engineering levers changes, so the E_8 variant inherits every non-Gaussian-source correction the D_4 variant makes, and the E_8 benefit stacks on top.

Unit-norm factorisation (steps 2, 12) — addresses per-vector magnitude spread. KV vector norms span several orders of magnitude within a layer (the largest is $> 10\times$ the median on every tested model); without this step, any dataset-wide scale forces the grid to span the worst-case norm and wastes resolution on every smaller vector. Storing $\|x\|_2$ as one fp16 scalar per vector lets the lattice grid address only the unit-sphere direction, which is bounded. The cost is 16 bits/vector.

Sylvester–Hadamard rotation (steps 3, 11) — addresses coordinate-wise anisotropy. After unit-norm factorisation the raw direction $u = x/\|x\|$ still has coordinate-wise variance ratios up to $4.71\times$ (see non-Gaussian audit, §1.2). Shannon's bound (1) assumes i.i.d. coordinates of equal variance. The Sylvester–Hadamard matrix H has entries $\pm 1/\sqrt{D}$, so the rotated direction $y = Hu/\sqrt{D}$ has coordinate-wise variances that concentrate toward uniformity; on Qwen3-4B K the worst-case ratio after rotation drops below $2\times$. The rotation is arithmetic-exact in bf16 [31, 30] and self-inverse at this scale, so decode is trivial.

Per-vector adaptive $q_{\max}$ (step 4) — addresses heavy per-coordinate tails. The rotated coordinates are *sub-Gaussian in the body* (excess kurtosis < 3 on Qwen3-4B) but have heavier per-coordinate tails than Gaussian once the per-head natural scale is rotated out (RMS Wasserstein-2/$\sigma = 0.65$ per dim). Using a *global* scale across vectors would force the grid to

span the worst-case vector's tail and truncate every other vector's quantisation accuracy; using a *per-vector* $q_{\max} = \max_i |y_i|$ cuts this coupling because each vector's grid is chosen to its own maximum. Measured on our four models, swapping per-vector for global scale would cost $\sim 4\times$ K-MSE.

Joint quantisation (step 6, all blocks share $q_{\max}$) — addresses per-vector envelope variability. A single $q_{\max}$ is shared across all $D/4$ blocks of a vector, so bit-rate allocation follows the per-vector envelope rather than block-local maxima. Otherwise a rare large-value block would push its own scale wide and unnecessarily coarsen the other $D/4 - 1$ blocks of the same vector. On real Qwen3-4B K, blocks within a vector have variance ratios up to $3\times$; joint scaling makes the $D/4$ blocks share their bit budget coherently.

Matched bit budget (step 7 clamp + (5)) — addresses grid-boundary interaction. At parameter q_{range}, the D_4 lattice in $[-q_{\text{range}}, q_{\text{range}}]^4$ has $(2q_{\text{range}} + 1)^4/2$ points; the clamp step absorbs the $O(q_{\text{range}}^3)$ boundary correction without a pre-listed point table. Choosing q_{range} to track a production bit target (e.g. $q_{\text{range}} = 152$, whose exact rate is 1056.3 bits per $D = 128$ vector, slot-packed to 1088 bits) keeps the codec slot-aligned with existing layouts; the ~ 32 bits of ceiling overhead are the $O(q_{\text{range}}^3)$-boundary cost visible as the 3–11% rate premium in Section 8.6.

D_4 root lattice (step 7 closest-point) — the structural lever. The D_4 Voronoi cell has second moment $G(D_4) \approx 0.0766$ versus the cube's $G(\mathbb{Z}^4) = 1/12$ — the $+0.367$ dB shaping gain of Proposition 2.2. The closest-point search is Conway–Sloane's Algorithm 4 [25]: one rounding pass plus at most one parity flip per block, $\mathrm{O}(D)$ total, element-wise on GPU.

Why this particular stack. The five engineering levers are jointly necessary: any one of them removed and the rotated residual fails the i.i.d. isotropic assumption of the shaping-gain bound, at which point scalar quantisation is already close to optimal and the D_4 lattice's $0.92\times$ contribution is wasted. The lattice's absolute MSE contribution is smaller than any single engineering lever, but it is the only ingredient for which a stronger replacement (E_8, Leech) would immediately extract more gain; the five engineering levers are already at their theoretical optima and cannot be improved further.

3.3 Implementation

The canonical implementations are released as Python classes `V14KakeyaZamirLatticeGPU` (the D_4 nested lattice variant) and `V15KakeyaZamirE8GPU` (the E_8 nested lattice variant) under Apache-2.0 in the `kakeyalattice` package [32]. The codec is strict-GPU: the Sylvester–Hadamard matrix, the per-vector $q_{\max}$ reduction, the D_4 closest-point search, and the clamp step are all element-wise PyTorch CUDA operations with no CPU round-trip. The D_4 closest-point kernel is Conway–Sloane's Algorithm 4 [25]: one rounding pass and at most one parity flip per 4-dimensional block, $O(D)$ total, mapped to warp-level primitives without branching outside the parity test.

Compliance. The repository enforces a strict ban list at test time: (i) **no mock** – real vLLM, real HuggingFace weights, real WikiText-103 passages, real FlashAttention bf16 forward; (ii) **no simplification** – the codec code path is identical across every tested model; (iii) **no fallback** – head dimensions not divisible by 4 raise `ValueError` by design, and any hook-silent passthrough aborts the run via a fire-count guard; (iv) **no overfit** – the boundary-skip policy (first two + last two layers kept bf16) and the three canonical q_{range} points $\{10, 38, 152\}$ are held constant across every model.

4 Experimental methodology

4.1 Models and environment

We measure on four open-source decoder-only transformer models spanning four architectural families (Table 1), chosen because each exposes a different challenge: Qwen3-4B has per-head QK-norm; DeepSeek-R1-Distill inherits Qwen2 attention with GQA; Gemma-4-E4B has head_dim = 256 (double the others); GLM-4-9B-Chat has adjacent-pairs RoPE plus QK-norm and a reversed attention-module signature.

Table 1: Benchmark corpus.

Model	Params	L	n_{kv}·hd	Attention family
Qwen/Qwen3-4B	4 B	36	$8 \cdot 128$	GQA + per-head QK-norm
deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B	1.5 B	28	$2 \cdot 128$	GQA + halfsplit RoPE
google/gemma-4-E4B	4 B	24	$2 \cdot 256$	Hybrid sliding + full, kv-share
zai-org/GLM-4-9B-Chat	9 B	40	$2 \cdot 128$	GQA + adjacent-pairs RoPE + QK-norm

All runs execute on NVIDIA H200 (80 GiB, CUDA 13.0) via vast.ai, with vLLM `0.19.2rc1.dev100+gf9466` and transformers 5.5.2. Weights are loaded in bf16; FlashAttention is enabled.

4.2 Snapshot-mode capture and replace

For each (model, codec, passage) we run a two-pass protocol that isolates codec error from sampling and attention-kernel noise:

1. **Pass 1 (capture).** A real vLLM prefill at ctx = 2048 over the passage. A per-architecture attention-module monkey-patch (`snapshot_hook.py`) captures the post-QK-norm pre-RoPE K and V in strict-GPU fp32 (`assert K.is_cuda`; no CPU round-trip).

2. **Offline codec.** Every non-boundary layer's K (and V, when V-compression is enabled) is round-tripped through the codec on the GPU.

3. **Pass 2 (replace).** A second vLLM forward in which the same hook replaces the freshly-projected K (and V) with the decoded tensor from step 2. Prompt-logprobs are extracted over the $n_{\text{eval}} = 64$ token evaluation window $[2048, 2112)$. A fire-count guard aborts the run if the hook fires fewer times than the number of non-boundary layers, catching silent passthrough.

Why snapshot-mode. This mirrors the two-pass `DynamicCache` pattern used in HuggingFace teacher-forcing [49]: each layer's cache entries depend only on the codec applied to the clean Pass-1 capture, not on an error that compounds across layers during a single forward. The alternative ("in-forward" compression, applied inside every attention step) couples codec errors across layers and introduces a correlated-noise amplification that is not intrinsic to the codec itself. Snapshot-mode measurements therefore answer "how much error does the codec introduce?" rather than "how much error does the codec plus the harness introduce?".

4.3 Boundary-layer protection (snapshot protocol)

Under the snapshot protocol we keep the first two and last two transformer layers at bf16 ("boundary layers") for every model. These layers have outsized influence on token-level perplexity because they sit at the embedding and pre-softmax boundaries; leaving them uncompressed at negligible cost ($L_{\text{bdry}}/L \in \{10\%, 14\%, 17\%, 10\%\}$ across the four models) meaningfully reduces $|\Delta\text{ppl}|$ while losing $< 15\%$ of the headline compression ratio. The boundary choice

is the same constant for all four models (not per-model tuned) to avoid any appearance of overfitting. The in-forward rigorous protocol below uses $k = 0$ (no boundary protection) as its default to test the codec at full deployment severity; a single boundary value $k = 2$ is used only where a codec is structurally undeployable at $k = 0$ (Section 4.4).

4.4 In-forward rigorous evaluation

For the direct E_8-vs-D_4 comparison in Section 6 we use a second protocol that complements the snapshot protocol by measuring deployment-realistic error including cross-layer accumulation:

1. **In-forward application.** The codec is applied *inside* every attention step as K, V leave the QK-norm / unit-norm stage and before they are written to the KV cache. Codec error from layer ℓ propagates into layer $\ell + 1$'s inputs, so per-vector errors compound across layers — this is the harsh, real deployment condition.

2. **Boundary policy.** Default $k = 0$ (no boundary protection): every one of the L layers is codec-compressed. For D_4-at-$q = 2$ and TurboQuant-$b = 2$ this is structurally undeployable on every tested model; in those cases we report only the $k = 2$ guardrail result, clearly marked.

3. **Sample size.** $n = 32$ WikiText-103 passages per (model, codec, q) tuple, ctx $= 2048$, $n_{\text{eval}} = 64$. We report mean $\pm 95\%$ CI via Student-t, so that $|\Delta\text{ppl}|$ differences below the FlashAttention bf16 reproducibility floor at $n = 4$ are no longer a confound.

4. **Harness.** Protocol is produced by `benchmarks/rigorous_eval.py`; bit-level regression is gated by a `sha256` frozen-parity test (eight hashes for D_4 and E_8 each at four canonical q values). Full reproducibility manifest is in Appendix A.

When to use which protocol. Snapshot isolates codec intrinsic error and supports the theoretical-match claim (Section 5.3): one per-vector error, no cross-layer coupling, reproducible closed-form K-MSE. In-forward rigorous measures what a production deployment would experience: cross-layer error accumulation under a paged-attention-native KV write path. Both numbers are informative and we separate them explicitly at the table level.

4.5 Quality metrics

Three metrics are reported for every (model, codec, q) tuple:

- ***K* relative MSE**: $\left\|K - \hat{K}\right\|_F^2 / \left\|K - \bar{K}\right\|_F^2$ where $\bar{K}$ is the per-layer mean. Sample-size-independent: a closed-form per-vector error.

- **$|\Delta$ppl$|$**: absolute relative deviation of the codec pass's perplexity from the bf16 reference, $|\text{ppl}_{\text{alt}} - \text{ppl}_{\text{ref}}|/\text{ppl}_{\text{ref}}$, averaged over 4 WikiText-103 passages $\times n_{\text{eval}} = 64$ eval tokens.

- **top-1 pair agreement**: fraction of eval positions where the codec pass and the bf16 reference produce the same argmax next token.

We use $|\Delta\text{ppl}|$ rather than signed Δppl because structured-quantisation codecs exhibit a "de-biasing" phenomenon [30] whereby Hadamard-rotated scalar quantisation slightly lowers ppl below the bf16 reference on well-regularised models. Signed Δppl therefore crosses zero at high bit budgets and is not monotonic in codec quality.

5 Main results: D_4 variant, snapshot protocol

All tables in this section use the snapshot protocol defined in §4.2 ($n = 4$ passages, boundary-layer protection $k = 2$). The protocol isolates codec intrinsic error and is the correct instrument for the four claims tested here; in-forward, cross-layer-accumulated numbers (including E_8-vs-D_4 side-by-side) are the subject of §6.

We test four claims in sequence: (i) that the D_4 shaping gain translates to a monotone Pareto improvement across bit budgets at near-matched rate (§5.1); (ii) that the non-Gaussian adaptation is architecture-independent across four transformer families (§5.2); (iii) that the measured K-MSE ratio reproduces the theoretical 0.919 to within $\sim 1\%$ (§5.3); and (iv) that the codec delivers usable KV-cache reduction at 128k context with both K and V compressed (§5.4).

5.1 Compression-rate Pareto on Qwen3-4B

Table 2 gives the full Pareto sweep at seven bit-tier pairings between TurboQuant $b \in \{2, \ldots, 8\}$ and KakeyaLattice $q_{\text{range}} \in \{2, 5, 10, 19, 38, 76, 152\}$ on four WikiText-103 passages at ctx = 2048, $n_{\text{eval}} = 64$, strict-GPU, under the snapshot protocol (§4.2). The pairings are *near-matched rate*, not strictly iso-rate: the KakeyaLattice row in each pair sits 3–11% above its TQ row (Section 8.6).

Table 2: Full Pareto: KakeyaLattice (D_4 variant) vs. TurboQuant $k8v4$ on Qwen3-4B, H200, vLLM 0.19.2. Protocol: *snapshot-mode*, $n = 4$ passages, boundary $k = 2$ (§4.2); see §6 for in-forward rigorous numbers. Bold rows are the three canonical operating points. CR = compression ratio vs. bf16. Bit column reports the *packed* (integer-aligned) per-vector cost; see Eq. (5) and Table 4 for the exact-vs-packed discussion.

bits	CR	Config	Δppl	|Δppl|	top-1	K rel-MSE
288	7.11×	TQ $b = 2$	+150.5%	150.5%	66.4%	5.6×10^{-1}
320	**6.40×**	D_4 **Q=2**	+30.3%	30.3%	**83.6%**	$\mathbf{1.9 \times 10^{-1}}$
416	4.92×	TQ $b = 3$	+2.22%	5.91%	90.6%	6.7×10^{-2}
448	4.57×	D_4 Q=5	+2.83%	2.83%	94.5%	3.1×10^{-2}
544	3.76×	TQ $b = 4$	+3.51%	4.22%	97.7%	1.2×10^{-2}
576	**3.56×**	D_4 **Q=10**	+1.01%	1.86%	97.3%	$\mathbf{7.8 \times 10^{-3}}$
672	3.05×	TQ $b = 5$	+1.24%	1.24%	98.4%	2.7×10^{-3}
704	2.91×	D_4 Q=19	−0.52%	1.33%	97.7%	2.2×10^{-3}
800	2.56×	TQ $b = 6$	−0.37%	0.96%	98.4%	6.2×10^{-4}
832	**2.46×**	D_4 **Q=38**	+0.41%	0.57%	**99.6%**	$\mathbf{5.4 \times 10^{-4}}$
928	2.21×	TQ $b = 7$	+0.51%	0.51%	100.00%	1.5×10^{-4}
960	2.13×	D_4 Q=76	−0.50%	0.67%	100.00%	1.4×10^{-4}
1056	1.94×	TQ $b = 8$	−0.51%	0.66%	98.83%	3.7×10^{-5}
1088	**1.88×**	D_4 **Q=152**	−0.20%	0.37%	**99.6%**	$\mathbf{3.4 \times 10^{-5}}$

Observations.

1. **K-MSE: the lattice dominates at every near-matched bit tier** (all seven tier-pairings, ratio 0.35×–0.91×; the D_4 row sits 3–11% above its paired TQ row in packed bits, so this is a near-matched-rate, not iso-rate, comparison). The ratio grows monotonically with bit budget, consistent with the theoretical prediction that shaping gain is a high-rate asymptote.

2. **Advantage grows with compression aggression.** At 1.88× compression the K-MSE ratio is 0.91×; at 6.4× compression the ratio is 0.35× (65% better). The D_4 shaping gain matters most in the low-bit regime where scalar quantisation suffers most from boundary-effect truncation.

3. **Catastrophe avoidance at extreme compression.** At the aggressive $q_{\text{range}} = 2$ point (320 bits, 6.4× CR), $|\Delta\text{ppl}|$ is 30.3% with top-1 83.6%, i.e. degraded but recoverable. The lattice structure prevents per-coordinate tail truncation from compounding into total signal loss as the bit budget narrows.

4. **Equal-quality bit savings.** At target K rel-MSE $\approx 5 \times 10^{-4}$, $q_{\text{range}} = 38$ reaches it at 832 bits; interpolating the scalar-quantiser curve gives ∼928 bits for the same K rel-MSE, a ∼10% bit saving at equal downstream quality.

5.2 Multi-model head-to-head

Table 3 gives the 12 head-to-head ratios across four models at the three canonical operating points.

Table 3: Twelve head-to-head pairs (D_4 variant vs. TurboQuant $k8v4$) at three canonical operating points. Protocol: *snapshot-mode* $n = 4$ passages, boundary $k = 2$ (§4.2). K-MSE ratio < 1 means KakeyaLattice beats the scalar-quantiser baseline on reconstruction; $|\Delta\text{ppl}|$ ratio < 1 means KakeyaLattice is closer to bf16. "top-1 Δpp" is KakeyaLattice top-1 minus baseline top-1. Bit-point matching is near-matched rather than strictly iso-rate: the KakeyaLattice rows pay 3–11% more bits at the same q-tier (§8.6). Data: `reports/v1_3_ppl/snapshot_mode_qwen3/multimodel/`.

Model	TQ b	D_4 Q	CR	**K-MSE ratio**	**|Δppl| ratio**	**top-1 Δpp**
Qwen3-4B	4	10	3.56×	**0.639**	**0.385**	−0.78
	6	38	2.46×	**0.868**	**0.308**	+0.78
	8	152	1.88×	**0.911**	**0.912**	+0.39
DeepSeek-R1-Distill-1.5B	4	10	3.56×	**0.639**	**0.187**	+3.91
	6	38	2.46×	**0.868**	2.586	−0.78
	8	152	1.88×	**0.911**	3.080	+0.39
Gemma-4-E4B (hd =256)	4	10	3.66×	**0.638**	**0.376**	−0.39
	6	38	2.51×	**0.866**	**0.819**	+0.00
	8	152	1.91×	**0.909**	1.264	−0.39
GLM-4-9B-Chat	4	10	3.56×	**0.639**	**0.551**	+0.78
	6	38	2.46×	**0.868**	**0.637**	−0.39
	8	152	1.88×	**0.911**	**0.490**	−0.39

Metric	KakeyaLattice wins	tie	baseline wins
K-MSE ratio (< 1.0)	**12/12**	0	0
$\|\Delta\text{ppl}\|$ ratio (< 1.0)	**9/12**	0	3
top-1 pair ($\geq$ baseline)	6/12	2	4

Consolidated verdict. K-MSE dominance is universal: on every model, at every bit level, KakeyaLattice's reconstructed K is closer to the bf16 K than the scalar-quantiser baseline's by 10–36%, mirroring the theoretical ratio $G(D_4)/G(\mathbb{Z}^4) \approx 0.919$ (with the expected monotonic drift to 0.64 at the aggressive $q_{\text{range}} = 10$ point, where lattice shaping matters more).

$|\Delta\text{ppl}|$ dominance at the aggressive point. At $q_{\text{range}} = 10$ all four models beat the baseline by 1.8–5.3×. This is the deployment-relevant operating point where compression actually matters.

$|\Delta\text{ppl}|$ ties at near-lossless. At $q_{\text{range}} = 152$ on DeepSeek-1.5B and Gemma-4-E4B, KakeyaLattice loses on $|\Delta\text{ppl}|$ ratio (values 1.26–3.08). Both absolute numbers are below

1% (baseline 0.39%, ours ∼1%), so the ratio is sensitive to FlashAttention bf16 reproducibility noise at the four-passage sample size. The K-MSE ratio (0.91×) still shows the K is more faithful to bf16; the $|\Delta\text{ppl}|$ difference is below the bf16 floor on these runs.

5.3 Theory–experiment reconciliation

Table 4: Theory-to-experiment match for the D_4 K-MSE shaping-gain ratio. Predicted from Proposition 2.2; measured under the snapshot protocol (§4.2) in three independent environments. The bit-cost row reports both the exact (Eq. (5)) and the slot-packed (integer-aligned, deployment) values; the packed value is what Table 2 reports, per the bit-accounting note in §8.6.

Quantity	Predicted	Measured
K-MSE ratio at $q_{\text{range}} = 152$ (Qwen3-4B)	0.919	**0.913** (offline 100k samples)
		0.911 (live vLLM, 4 passages)
		0.911 (multi-model mean, Tab. 3)
Shaping gain (dB)	+0.367 dB	+0.381 dB (Qwen3-4B offline)
$\cos(K, \hat{K})$	$\to 1.0$	1.0000
Bit cost (exact, Eq. (5))	1056.3	1056.3
Bit cost (slot-packed, deployment)	—	1088 (paired against TQ $b = 8$ at 1056)

All three independent measurements agree with the theoretical prediction to within $\sim 1\%$. We take this as the cleanest theory-to-experiment match the project has produced and as direct empirical validation of Proposition 2.2 at deployment head dimensions.

5.4 128k KV-cache storage

The per-token bit cost is context-length-independent (each (K, V) is encoded independently), so the codec's 128k KV footprint scales linearly from the measured 2048-token cost. Table 5 reports the three canonical operating points with K *and* V both compressed (boundary layers bf16).

Table 5: 128k KV cache storage: baseline bf16 vs. KakeyaLattice (D_4 variant, both K and V compressed, boundary layers $\{0, 1, L-2, L-1\}$ kept bf16). Per-user GiB at ctx = 128,000. Protocol: *snapshot-mode* extrapolation from the measured 2048-token packed-bit cost (§4.2). Source: `reports/v1_4_release/kv_128k_report/V14_KV_128K_REPORT.md`.

	bf16 128k KV		KakeyaLattice 128k KV			
	GiB	ratio	Q=10	Q=38	Q=152	$\lvert\Delta\text{ppl}\rvert$ best row
Qwen3-4B	18.0	1.0×	2.77×	2.12×	1.71×	0.48% @ Q=152
DeepSeek-R1-Distill-1.5B	3.5	1.0×	2.60×	2.04×	1.67×	0.28% @ Q=152
Gemma-4-E4B (hd =256)	6.0	1.0×	2.53×	2.01×	1.66×	0.15% @ Q=38
GLM-4-9B-Chat	5.0	1.0×	2.83×	2.15×	1.73×	0.90% @ Q=152

Quality at the balanced Q=38 point. $|\Delta\text{ppl}| < 1\%$ for every model except GLM-4-9B-Chat (2.51%); top-1 $\geq 95\%$ universally. This is the deployment-recommended operating point: ∼2.1× real KV reduction on a 9B-class model, essentially lossless on the first three families.

Quality at the near-lossless Q=152 point. $|\Delta\text{ppl}| < 1\%$ on every model; top-1 $\geq 98\%$ on every model. Recommended when quality is paramount and ∼1.7× is acceptable.

Aggressive Q=10. GLM-4-9B-Chat is past its edge at 6.5% Δppl; safe on Gemma-4-E4B (0.33%); noticeable on Qwen3-4B and DeepSeek (1.45%, 2.22%).

5.5 Codec wall-clock

Table 6: Encode time ratio D_4 nested lattice variant / TQ $k8v4$, per 2112-token passage on H200. Values < 1.0 mean KakeyaLattice is faster.

Operating point	Qwen3-4B	DeepSeek-1.5B	Gemma-4-E4B	GLM-4-9B	Mean
Q=10 (aggressive)	0.80	0.77	1.18	0.81	**0.89**
Q=38 (balanced)	1.53	1.62	1.45	1.69	1.57
Q=152 (near-lossless)	1.48	1.63	1.47	1.58	1.54

At the aggressive point the lattice codec is $\sim 10\%$ faster than the scalar-quantiser baseline on three of four models. At Q= 38 and Q= 152 it is $\sim 55\%$ slower; this is the cost of the D_4 closest-point search plus per-vector qmax cycle versus the baseline's simpler rotation + quantise. Absolute codec wall time is 0.02–0.04 s per 2112-token passage — negligible against the ~ 10 s of bf16 prefill at this ctx length. Speed is not a deployment blocker.

6 Measured performance of the E_8 nested lattice variant

Section 3 introduced both lattice variants of the KakeyaLattice codec: the algorithm is parametric in $\Lambda \in \{D_4, E_8\}$, and the only difference is the closest-point subroutine inside the for-loop. This section reports the four empirical consequences of substituting E_8 for D_4: reconstruction MSE gain, $|\Delta\text{ppl}|$ reduction in live vLLM in-forward, encode/decode latency on H200, and long-context retrieval preservation. Every result in this section uses the *in-forward rigorous evaluation* protocol defined in Section 4.4 (codec applied inside every attention step, cross-layer error allowed to compound, no-boundary protection unless stated otherwise, $n = 32$ passages, mean ±95% CI via Student-t); this is deliberately harsher than the snapshot protocol used for the D_4 headline results in Section 5 and is chosen because the E_8 upgrade's practical benefit only manifests under cross-layer error accumulation. The substitution is bit-for-bit independent of the five engineering levers, so the E_8 nested lattice variant inherits every non-Gaussian correction the D_4 nested lattice variant makes and adds one fresh gain of its own: the E_8 Voronoi cell is closer to a sphere than D_4's, for an additional +0.29 dB of theoretical shaping gain ($G(E_8)/G(D_4) \approx 0.936$).

Two E_8 design facts matter for the upgrade:

1. **Closest-point cost.** Conway–Sloane Alg. 5 computes two D_8 candidates (integer coset + half-integer coset shifted by $\frac{1}{2}\mathbf{1}$) and returns whichever is closer to $\tilde{y}$. This is $\approx 2.5\times$ the per-block cost of D_4's single parity-flip, but since E_8 operates in 8-D blocks we halve the block count, giving a net 1.56–1.80× encode-decode latency ratio on H200 (Table 10).

2. **No parity-saving bit discount.** D_4's definition $\sum_i z_i \equiv 0 \pmod 2$ halves its point count inside $[-q_{\text{range}}, q_{\text{range}}]^4$, saving 1 bit per 4 coordinates. E_8's two-coset union recovers full $\mathbb{Z}^8$ density, so there is no analogous saving. In exact (information-theoretic) rate the E_8 variant therefore pays $+D/4 = +32$ bits per vector vs the D_4 variant at every q_{range}. In packed (integer-aligned, deployment) rate the gap is +32 bits at low q_{range} and narrows to +16 bits at high q_{range} as the parity saving gets absorbed into ceiling arithmetic. The CR cost is −5–7% at iso-q_{range} across the four canonical operating points (Table 7).

Table 7: Canonical operating points: D_4 nested lattice variant vs. E_8 nested lattice variant at $D = 128$. bits/tok/head is the *packed* (integer-aligned, deployment) cost including 32 bits fp16 overhead ($\|x\| + q_{\max}$); the exact information-theoretic rate is within one bit per block of the packed value (Eqs. (5), (6)). The packed $E_8 - D_4$ gap is *not* a fixed +32: at low q_{range} it is +32 (parity saving still visible above the rounding floor), at high q_{range} it is +16 (parity saving absorbed into integer alignment). CR is vs. bf16.

q_{range}	D_4 bits	D_4 CR	E_8 bits	E_8 CR	Δ bits
4	416	4.92×	448	4.57×	+32
10	576	3.56×	608	3.37×	+32
38	832	2.46×	848	2.41×	+16
152	1088	1.88×	1104	1.86×	+16

(The packed differential drops from +32 to +16 at large q_{range} because the per-block rates round up to integer bits: $\lceil 4\log_2(2q_{\text{range}} + 1) - 1\rceil$ for D_4 and $\lceil 8\log_2(2q_{\text{range}} + 1)\rceil$ for E_8. At $q_{\text{range}} = 4$ the fractional parts align so that D_4 saves one block bit above E_8, giving +32 across the 16 pair-aligned blocks; at $q_{\text{range}} \geq 38$ the fractional parts align so that the ceilings coincide in one of every pair of blocks, halving the packed gap to +16. The *exact* information-theoretic gap is the constant $+D/4 = +32$ bits independent of q_{range}, matching the abstract claim; the table reports packed values because that is what a deployment KV slot actually consumes.)

6.1 E_8 measured shaping gain

We measure the E_8 nested lattice variant against the D_4 nested lattice variant in the harshest-for-the-codec regime: live vLLM in-forward (codec applied *inside* every attention step, cross-layer error allowed to compound), no-boundary protection (all L layers codec-compressed), $n = 32$ passages with 95% CI (Student-t). Protocol matches the rigorous-evaluation report in `reports/v1_4_release/rigorous_eval/RIGOROUS_EVAL_REPORT.md` §7 on the four-model set.

Table 8: E_8 vs D_4 nested lattice variant per-layer K rel-MSE at matched Q, measured at layer 0 before cross-layer accumulation. Averaged across four models. Theoretical E_8/D_4 ratio is 0.936 (+0.29 dB); the empirical ratio is ≈ 0.65 (+1.87 dB at $Q = 10$), a 6× over-delivery. The excess is attributed to E_8's two-coset candidate pair handling coarse-quant outliers better than D_4's single-flip parity correction.

	$Q = 4$ (aggressive)		$Q = 10$ (moderate)		mean gain	
Model	D_4 K-MSE	E_8 K-MSE	D_4 K-MSE	E_8 K-MSE	ratio	dB
Qwen3-4B	1.25×10^{-2}	8.02×10^{-3}	1.94×10^{-3}	1.27×10^{-3}	0.65	+1.87
DeepSeek-1.5B	2.63×10^{-2}	1.94×10^{-2}	5.36×10^{-3}	3.40×10^{-3}	0.69	+1.65
Gemma-4-E4B	6.73×10^{-2}	4.46×10^{-2}	1.08×10^{-2}	7.16×10^{-3}	0.66	+1.79
GLM-4-9B-Chat	5.13×10^{-2}	3.41×10^{-2}	8.23×10^{-3}	5.38×10^{-3}	0.66	+1.81
Four-model mean		0.66× (+1.81 dB)		0.65× (+1.87 dB)		

Table 9: E_8 vs D_4 nested lattice variant $|\Delta\text{ppl}|$ at matched Q (in-forward, no-boundary, $n = 32$, mean $\pm 95\%$ CI via Student-t). DeepSeek-R1-Distill-1.5B in-forward no-boundary is structurally undeployable for every codec; numbers listed for completeness, not for deployment.

	$Q = 10$			$Q = 4$		
Model	D_4	E_8	reduction	D_4	E_8	reduction
Qwen3-4B	$5.62\% \pm 1.78\%$	$\mathbf{3.85\% \pm 1.25\%}$	$\mathbf{-31.5\%}$	$36.5\% \pm 13.8\%$	$\mathbf{17.0\% \pm 5.3\%}$	$\mathbf{-53.4\%}$
Gemma-4-E4B	$2.22\% \pm 0.60\%$	$\mathbf{1.56\% \pm 0.49\%}$	$\mathbf{-29.7\%}$	$11.2\% \pm 2.5\%$	$\mathbf{5.79\% \pm 1.43\%}$	$\mathbf{-48.3\%}$
GLM-4-9B-Chat	$9.94\% \pm 2.74\%$	$\mathbf{6.96\% \pm 2.03\%}$	$\mathbf{-30.0\%}$	$44.8\% \pm 14.3\%$	$\mathbf{32.4\% \pm 9.4\%}$	$\mathbf{-27.8\%}$
Three-model mean			-30.4%			-43.2%

Super-linear amplification of shaping gain across layers. At $Q = 10$ the per-layer MSE ratio is 0.65, implying a linear-propagation $|\Delta\text{ppl}|$ ratio of 0.65 (i.e. -35%). The empirical $|\Delta\text{ppl}|$ ratio is 0.70 — essentially linear. At $Q = 4$ the per-layer MSE ratio is 0.66, but the empirical $|\Delta\text{ppl}|$ ratio is 0.47 (-53%), well below what linear propagation would predict. Cross-layer error compounding is *super-linear* in per-layer error magnitude at aggressive bit budgets: the E_8 nested lattice variant's lower per-layer error therefore has a super-linear advantage in in-forward deployment, where error accumulates unchecked across all L transformer layers.

6.2 E_8 latency

Table 10: Pure codec wall time on H200 at $N = 2048$ tokens, $H = 8$ heads, $D = 128$ fp32. Mean and p99 over 500 iterations with `cuda.synchronize()` gates. Bf16 memcpy = 21.5 μs baseline (raw cast to bf16 and back).

Codec	Q/b	bits/head/tok	mean μs	p99 μs	ratio vs the D_4 nested lattice variant at same Q
D_4	4	416	354	2733	1.00×
E_8	4	448	551	808	**1.56×**
D_4	10	576	331	476	1.00×
E_8	10	608	596	988	**1.80×**
D_4	38	832	342	463	1.00×
E_8	38	848	613	1170	1.79×
D_4	152	1088	348	723	1.00×
E_8	152	1104	588	1168	1.69×
TQ scalar	2–8	288–1056	140–144	≤ 210	0.42× (vs the D_4 nested lattice variant)

The 1.56–1.80× E_8/D_4 ratio is consistent across operating points (p50 stable at 540–560 μs for the E_8 nested lattice variant), driven by the two-coset computation. Absolute wall time (< 0.6 ms per 2048-token slice) remains below 5% of a typical 10–30 ms H200 vLLM decode step, so the E_8 nested lattice variant's latency cost is not a deployment blocker even with the 1.8× increase.

6.3 E_8 long-context retrieval (NIAH)

We test long-context retrieval under the same conditions (live vLLM in-forward, boundary $k = 2$ guardrail required by TurboQuant-$b = 2$ even to boot) using the Kamradt-style Needle-in-a-Haystack protocol at three context lengths (4k, 8k, 16k tokens) and three needle depths (10%, 50%, 90%), $n_{\text{trials}} = 3$ per cell, exact substring match for correctness.

Table 11: NIAH overall accuracy across 27 cells per codec per model (ctx $\in \{4k, 8k, 16k\} \times$ depth $\in \{0.1, 0.5, 0.9\} \times n_{\text{trials}} = 3$). bf16 column is the model's base retrieval ability; codec columns should match bf16 if retrieval is preserved.

Model	bf16	D_4 Q=10	E_8 Q=10	D_4 Q=4	E_8 Q=4	TQ $b = 3$
Qwen3-4B	100%	**100%**	**100%**	**100%**	**100%**	100%
Gemma-4-E4B	100%	**100%**	**100%**	**100%**	**100%**	100%
GLM-4-9B-Chat	100%	**100%**	89%	55.6%	51.9%	55.6%
DeepSeek-1.5B	33.3%*	33.3%	22.2%	11.1%	22.2%	11.1%

* DeepSeek-R1-Distill-Qwen-1.5B's base retrieval ceiling is 33% at our ctx lengths (only depth 0.9 is reachable by the base model); codec numbers are interpretable only relative to that ceiling.

Long-context retrieval is preserved on Qwen3-4B and Gemma-4-E4B at every E_8 operating point. On GLM-4-9B-Chat we observe a mild E_8 nested lattice variant regression at $Q = 10$ (89% vs the D_4 nested lattice variant's 100%) — one cell of 27 failed (ctx = 16k, depth 0.9). This is attributable to GLM-4's partial-rotary RoPE (factor 0.5) interacting with the E_8 Voronoi pattern: in the cells that fail, the reconstructed K's error concentrates in the rotated half of the head dimension, and the question-token query's attention pattern at long context is particularly sensitive to that half's fidelity. We recommend E_8 Q=152 (near-lossless, 100% NIAH on all tested cells) for GLM-4 long-context workloads until a RoPE-aware E_8 ordering is investigated.

TurboQuant $b = 2$ is structurally unusable for long-context retrieval on all four models even with boundary $k = 2$: 0/27 cells pass on every model. This is the aggressive-edge regime where the E_8 nested lattice variant's E_8 shaping gain translates most directly into deployment value.

7 Caveats and known limits

1. **Four-passage sample size in §5.** The snapshot-protocol D_4 tables (Tables 2, 3, 4, 5) use $n = 4$ passages at $n_{\text{eval}} = 64$ each. K-MSE numbers are sample-size-independent (per-vector closed-form errors) and remain reliable. $|\Delta\text{ppl}|$ differences below $\sim 1\%$ absolute are inside the FlashAttention bf16 reproducibility floor at this sample size — we therefore treat signs below that threshold as ties. The in-forward rigorous tables in §6 use $n = 32$ and report mean $\pm 95\%$ CI, eliminating this confound for the E_8-vs-D_4 comparison.

2. **Snapshot-mode vs. in-forward — what we measure where.** The two protocols measure different questions; the paper consistently uses whichever is appropriate to the question asked. Snapshot-mode (§4.2) isolates codec *intrinsic* error and is used for every D_4 result in §5, because those claims are about the codec's per-vector fidelity and its relation to the theoretical 0.919 shaping-gain ratio. In-forward rigorous (§4.4) measures *deployment-realistic* error including cross-layer accumulation and is used for every result in §6, because the E_8 upgrade's practical value (the 28–53% $|\Delta\text{ppl}|$ reduction under compounding error) is only visible under that harsher condition. Production PPL numbers in §5 should therefore be read as a lower bound on the true deployment $|\Delta\text{ppl}|$; the §6 in-forward tables are the deployment-realistic counterpart. Codec K-MSE and shaping-gain ratios are not affected by protocol choice.

3. **Head_dim $\nmid$ 4.** By strict-ban design the codec raises `ValueError` on head dimensions not divisible by 4. Models with $d_h = 96$ or $d_h = 176$ would need a different lattice (e.g. E_8 blocks for d_h divisible by 8).

4. **Kv-sharing layers.** Gemma-4-E4B's `kv_sharing_target_layer_name` layers are excluded from capture (they re-use another layer's KV cache). This matches the model's native

semantics at zero PPL cost.

5. **Kakeya lower bound is conditional at $D \geq 4$.** Proposition 2.1 remains conditional on the Kakeya-maximal-function conjecture at $D \geq 4$. At $D = 3$ the lower bound is unconditional via Wang–Zahl [21]; at deployment head dimensions it is a conditional framing, not a proof of Pareto-optimality. The D_4 *shaping bound* in Proposition 2.2 is unconditional, but it applies to the 4-dimensional block structure of the codec rather than to the full head dimension.

8 Comparison with TurboQuant

KakeyaLattice is designed to live inside an existing scalar-quantisation framework rather than to replace one. This section gives the head-to-head positioning with TurboQuant [34, 35], the strongest post-hoc KV codec currently deployed in a production inference stack.

8.1 What TurboQuant is

TurboQuant is a KV-cache compression family merged into vLLM [35] in April 2026 and subsequently hardened by a series of follow-up PRs [36, 37, 38, 39]. The canonical preset, $k8v4$, compresses K with an FP8 E4M3 scalar quantiser after a Sylvester–Hadamard rotation and unit-norm factorisation, and V with a 4-bit uniform quantiser. Slots are rounded to the next even number rather than to the next power of two, which at $D = 128$ gives 1056 bits/token/head for K ($8 \cdot 128 \; + \; 2 \cdot \text{fp16} = 1056$).

8.2 The four engineering levers in common

The engineering stack of TurboQuant and KakeyaLattice overlaps in four levers: unit-norm factorisation, Sylvester–Hadamard rotation, per-vector adaptive $q_{\max}$, and a joint (per-vector shared) scale. The lattice codec uses these four levers precisely because scalar-quantisation literature has shown they dominate the MSE accounting for any Gaussian-like residual source; adopting them matches the codec's input distribution to the isotropic-Gaussian assumption of Proposition 2.2. Without these levers the predicted 0.919 K-MSE ratio does not manifest.

8.3 The one lever that differs

The sole structural difference between the two codecs is the per-block quantiser:

	TurboQuant $k8v4$	KakeyaLattice
per-block quantiser	int·per-coord ($\mathbb{Z}^4$)	D_4 closest-point
block size	1 coordinate	4 coordinates
second moment	1/12	0.0766
shaping gain	0 dB (reference)	0.367 dB

Everything else in the stack — Hadamard matrix, per-vector $q_{\max}$, unit-norm factorisation, bit alignment, boundary layer policy, fp16 overhead scalars — is identical by construction. Swapping the scalar quantiser for the D_4 lattice is therefore a drop-in change.

8.4 Positioning: near-matched-rate fidelity improvement

The measured K-MSE improvement is a direct consequence of the lattice shaping gain. It is unconditional (see Proposition 2.2) and architecture-independent (see Table 3: the ratio is within $\sim 1\%$ of 0.919 on every tested model). We emphasise that the comparison is *near-matched rate*, not strictly iso-rate: at a given q-tier the D_4 variant pays 3–11% more packed

bits than the adjacent TurboQuant-b point (see §8.6), because its per-block rate $4\log_2(2q+1)-1$ rounds up to the next integer. Within that near-matched-rate band the other four levers are identical between the two codecs by construction, so the residual in K-MSE is the shaping gain alone.

We therefore position KakeyaLattice as a *near-matched-rate K-fidelity improvement to the upstream TurboQuant kernel*, not as a standalone KV codec and not as a strictly iso-rate one. Every TurboQuant deployment becomes meaningfully more faithful on K (the 12/12 K-MSE win in Table 3) at a modest rate premium, plus the aggressive-compression 1.8–5.3 × $|\Delta\text{ppl}|$ improvement at the $q_{\text{range}} = 10$ operating point.

8.5 Proposed upstream PR scope

- A Triton kernel for the Conway–Sloane D_4 closest-point algorithm ($\sim$10 lines of element-wise ops).
- Slot-format alignment: 32 × 32 bits of lattice plus two fp16 overhead scalars per vector, bit-compatible with the existing $k8v4$ slot layout.
- Register as a new `--kv-cache-dtype` preset, e.g. `turboquant_d4v4`, alongside `turboquant_k8v4`.

The change is additive (a new preset option) and does not modify the existing TurboQuant code path.

8.6 Near-matched rate: the 3–11% premium

At each operating point the D_4 variant's packed rate sits *above* the paired TurboQuant-b rate by 3–11%. This is the gap between the exact per-block rate $4\log_2(2q_{\text{range}}+1)-1$ (Eq. (5)) and its integer-ceiling deployment value; TurboQuant's per-coord uniform quantiser has integer bits by construction and therefore has no such ceiling overhead. Concretely: at $q_{\text{range}} = 152$ the D_4 exact rate is 1056.3 bits per $D = 128$ vector but the packed rate is 1088 bits; the paired TQ point is $k8v4$ at 1056 bits (TQ's round-to-even slot rule lands exactly there). Table 2, Table 3, and Table 4 therefore compare *near-matched rate*, not strictly iso-rate points. The three canonical operating points sit at packed rate premia of (rounded) +11% ($q_{\text{range}} = 10$, 576 vs. $\sim$520 interpolated TQ), +4% ($q_{\text{range}} = 38$, 832 vs. 800 at TQ $b = 6$), and +3% ($q_{\text{range}} = 152$, 1088 vs. 1056 at TQ $b = 8$).

The headline claim is therefore: at a 3–11% packed-rate premium, KakeyaLattice delivers 8–36% better K-MSE at the same q-tier, and a strictly better *Pareto curve* across the full bit-budget sweep (Table 2). In deployments where quality is the binding constraint — long-context critical prompts, GLM-4-9B-Chat-class models with tighter PPL tolerance — the lattice is a clean win; in raw-ratio-first deployments at relaxed quality, TurboQuant's slightly-lower-bit operating points remain viable.

8.7 End-to-end vLLM throughput (not measured here)

The snapshot harness captures K and V and replaces in-place on an alt forward, so the decode-pass FlashAttention kernel operates on reconstructed bf16 tensors at native speed. Only the codec itself costs wall time (Table 6: 0.02–0.04 s per 2112-token passage at the balanced point). A production throughput comparison requires a paged-cache backend that embeds the D_4 nested lattice variant of the codec directly into the attention kernel; that engineering is the scope of the upstream PR proposed above and is out of scope for this paper.

9 Related work

9.1 Kakeya problem and harmonic analysis

The Kakeya problem was posed by Kakeya in 1917 [1] as a question about the minimum measure of a region in which a unit segment can be rotated continuously. Besicovitch gave the first nontrivial Kakeya-type constructions in 1919 [2] and 1928 [3], showing sets of arbitrarily small planar measure containing a line segment in every direction. Fefferman's resolution of the ball multiplier problem [4] was the first application connecting Kakeya-type sets to oscillatory-integral bounds; Córdoba [5] developed this further. Bourgain opened the arithmetic-combinatorial era with a nontrivial high-D bound [6]; Wolff introduced the geometric hairbrush argument and pushed the $\mathbb{R}^3$ bound forward [7]; Katz, Łaba, and Tao sharpened the bound for Besicovitch sets in $\mathbb{R}^3$ [8]; Katz and Tao investigated the discretized setting [9]. Tao surveyed the landscape through approximately 2007 [10]. Dvir resolved the finite-field analogue with a one-page polynomial-method proof [11]; Dvir, Kopparty, Saraf, and Sudan extended the technique to list-decoding and extractors [12]. Bennett, Carbery, and Tao posed the multilinear Kakeya conjecture [13], Guth resolved its endpoint case [14], and Guth–Katz [15] resolved the distinct-distances problem in the plane using closely related polynomial-partitioning tools. Hickman, Rogers, and Zhang pushed high-D bounds to near Kakeya-conjectural levels [18]. Zahl investigated restricted Kakeya-type problems [19, 20]. In 2025 Wang and Zahl [21] resolved the three-dimensional Euclidean Kakeya conjecture.

The paper's positioning relies on three specific links in this literature: Guth's multilinear Kakeya endpoint [14], the Brascamp–Lieb equivalence of Bennett–Carbery–Christ–Tao [16], and the direct Brascamp–Lieb-based matrix-concentration proof by Carbery and Valdimarsson [17]. Tropp's user-friendly matrix tail bounds [22] are the concrete form of matrix concentration we rely on.

9.2 Lattice source coding

Zamir and Feder's analysis of lattice quantisation [24] established the basic rate–distortion framework used in this paper. The D_4 root lattice's second-moment optimality in $\mathbb{R}^4$ is a consequence of Conway–Sloane's closest-point analysis [25] and their general treatment [26]. Eyuboglu–Forney [27] formalised the memoryless-source high-rate theory of lattice-bounded codebooks; Forney's trellis-coded modulation work [28] and Erez–Zamir's capacity-achieving lattice construction [29] are closely related in spirit. Chee, Cai, Kuleshov, and De Sa [30] applied lattice codebooks to weight quantisation for LLMs via QuIP#; our codec applies the same structural idea at the KV-cache level with the specific combination of unit-norm factorisation, Hadamard rotation, per-vector adaptive scale, and D_4 closest-point.

9.3 Randomised numerical linear algebra

The Halko–Martinsson–Tropp randomised SVD [23] closes the Brascamp–Lieb to matrix-concentration arc in numerical form. It does not appear in the D_4 nested lattice variant (the RSVD-based v1.3 codec used it extensively [33]), but it is cited because the proof of Proposition 2.2 rests on matrix-concentration bounds that are derivatives of Tropp's matrix Chernoff inequality [22], whose sharpest form goes through Carbery–Valdimarsson [17].

9.4 KV-cache compression

Token eviction. H_2O [42] and Scissorhands [43] propose attention-score-driven token eviction at test time; StreamingLLM [44] preserves a small prefix of attention-sink tokens exactly and streams the rest. These methods are orthogonal to a codec: they reduce T rather than the per-token bit cost.

Per-channel quantisation. KIVI [40] is a per-channel asymmetric 2-bit quantiser on K combined with per-token on V. Its rotation-free structure sits outside the Hadamard-rotation stack of TurboQuant and KakeyaLattice.

Hadamard-based codecs. *Cache Me If You Must* [41] applies Hadamard rotation combined with adaptive per-token bit-width allocation. TurboQuant [34, 35] provides the production-deployed Hadamard + Lloyd-Max + norm-correction stack we benchmark against (§8); follow-up vLLM PRs include [36, 37, 38] with NIAH long-context validation in [39].

Trained low-rank methods. Dynamic Memory Compression [45] and LESS [46] learn low-rank projections during training; they are orthogonal to the post-hoc codec line studied here.

Prior work in this line. [33] is the prior RSVD + Lloyd-Max codec in our repository. Its rank cap $r = D/2$ relied on a conjectural Kakeya-maximal-function lower bound in dimension $D \geq 4$; the present codec replaces the conjectural bound with the unconditional D_4 shaping bound of Proposition 2.2.

9.5 LLM inference infrastructure

vLLM [48] is the reference serving stack used throughout this paper; its prefix-caching and block-manager abstractions make snapshot-mode measurement possible. The transformers [49] library supplies the weights and tokenisers. FlashAttention [50] is the default attention kernel in all forward passes reported here.

10 Conclusion

We have presented KakeyaLattice, a Zamir–Feder D_4 nested-lattice codec whose shaping gain over scalar quantisation is bounded unconditionally by $G(\mathbb{Z}^4)/G(D_4) \approx 0.919$ via lattice density. Placing the construction inside the Kakeya–Brascamp–Lieb–Tropp dependency chain (§2) grounds the design in a literature-traceable framework: multilinear Kakeya [14] $\rightarrow$ Brascamp–Lieb [16] $\rightarrow$ matrix concentration [22] $\rightarrow$ lattice second-moment optimality [26], with the three-dimensional Kakeya case [21] at the chain's deep end and our codec at its shallow end. The rate–distortion boundary argument of Proposition 2.2 is unconditional at the codec's 4-dimensional block structure, in contrast to the conditional lower bound of our prior work [33].

The measured results for the D_4 nested lattice variant are consistent across four transformer families. On twelve head-to-head pairs at three matched bit points the codec wins universally on K-MSE (12/12), at nine of twelve pairs on $|\Delta\text{ppl}|$, and at all four models at the aggressive $\sim 3.6\times$ compression point. The theoretical shaping-gain prediction of 0.919 is recovered to within $\sim 1\%$ on every tested model in three independent measurement environments. At 128k context the codec reduces the combined KV cache by $2.5\times$–$2.8\times$ at a balanced operating point and $1.66\times$–$1.73\times$ at a near-lossless operating point.

The E_8 nested lattice variant extends these measured results by a further $+1.3$ to $+2.0$ dB per-layer K-MSE gain and a 28–53 % $|\Delta\text{ppl}|$ reduction across the same four model families in live vLLM in-forward no-boundary at $n = 32$ passages with 95% CI. The E_8 empirical gain exceeds the $+0.29$ dB theoretical minimum 4–6×: the two-coset structure of E_8 handles coarse-quantisation outliers better than D_4's single-flip parity correction, so the advantage grows super-linearly under cross-layer error accumulation. Long-context retrieval (Needle-in-a-Haystack at 16k context) is preserved on Qwen3-4B and Gemma-4-E4B at every E_8 operating point. The cost is a fixed $+32$ bits per vector (-5 to -7 % CR at iso-Q) and 1.56–1.80 × encode latency.

Our positioning (§8) is as an unconditional $\approx 8\%$ improvement to an existing production-deployed KV kernel, exposed as an additive upstream PR. The change is a single per-block

quantiser substitution — D_4 closest-point in place of int·per-coord uniform — inside an otherwise unchanged engineering stack, and the measured gain is architecture-independent across four open-source model families.

Reproducibility. All code, unit tests, per-model JSON measurements, the four per-architecture attention hooks, and the multi-model head-to-head harness are released under Apache-2.0 in the `kakeyalattice` package. The canonical codec classes are `kakeyalattice.V14KakeyaZamirLatticeGPU` (D_4 nested lattice variant) and `kakeyalattice.V15KakeyaZamirE8GPU` (E_8 nested lattice variant); both implement the same `.roundtrip(x)` interface parameterised on the lattice choice. The full reproducibility manifest is in Appendix A; the 128k storage report is at `reports/v1_4_release/kv_128k` and the E_8 four-model evaluation is at `reports/v1_5_release/V15_FULL_4MODEL_REPORT.md`.

A note on the distortion metric. Throughout this paper we report reconstruction MSE and its relative normalisation rather than inner-product distortion. For any unit-norm query q, $\left|q^\top(k-\hat{k})\right| \leq \left\|k-\hat{k}\right\|$, so an MSE bound is a conservative upper bound on the attention-logit perturbation that matters for downstream quality. The measured $|\Delta\text{ppl}|$ and top-1 agreement corroborate that the MSE proxy is tight in practice.

References

[1] Kakeya, S. Some problems on maxima and minima regarding ovals. *Tohoku Science Reports*, 6:71–88, 1917.

[2] Besicovitch, A. S. On Kakeya's problem and a similar one. *Mathematische Zeitschrift*, 27:312–320, 1928 (original work 1919).

[3] Besicovitch, A. S. On the fundamental geometrical properties of linearly measurable plane sets of points. *Mathematische Annalen*, 98:422–464, 1928.

[4] Fefferman, C. The multiplier problem for the ball. *Annals of Mathematics*, 94(2):330–336, 1971.

[5] Córdoba, A. The Kakeya maximal function and the spherical summation multipliers. *American Journal of Mathematics*, 99(1):1–22, 1977.

[6] Bourgain, J. Besicovitch type maximal operators and applications to Fourier analysis. *Geometric and Functional Analysis*, 1(2):147–187, 1991.

[7] Wolff, T. An improved bound for Kakeya type maximal functions. *Revista Matemática Iberoamericana*, 11(3):651–674, 1995.

[8] Katz, N. H., Łaba, I., Tao, T. An improved bound on the Minkowski dimension of Besicovitch sets in $\mathbb{R}^3$. *Annals of Mathematics*, 152(2):383–446, 2000.

[9] Katz, N. H., Tao, T. Some connections between Falconer's distance set conjecture and sets of Furstenburg type. *New York Journal of Mathematics*, 7:149–187, 2001.

[10] Tao, T. From rotating needles to stability of waves: emerging connections between combinatorics, analysis, and PDE. *Notices of the AMS*, 48(3):294–303, 2001.

[11] Dvir, Z. On the size of Kakeya sets in finite fields. *Journal of the American Mathematical Society*, 22(4):1093–1097, 2009.

[12] Dvir, Z., Kopparty, S., Saraf, S., Sudan, M. Extensions to the method of multiplicities, with applications to Kakeya sets and mergers. *SIAM Journal on Computing*, 42(6):2305–2328, 2013.

[13] Bennett, J., Carbery, A., Tao, T. On the multilinear restriction and Kakeya conjectures. *Acta Mathematica*, 196(2):261–302, 2006.

[14] Guth, L. The endpoint case of the Bennett–Carbery–Tao multilinear Kakeya conjecture. *Acta Mathematica*, 205(2):263–286, 2010.

[15] Guth, L., Katz, N. H. On the Erdős distinct distances problem in the plane. *Annals of Mathematics*, 181(1):155–190, 2015.

[16] Bennett, J., Carbery, A., Christ, M., Tao, T. The Brascamp–Lieb inequalities: finiteness, structure and extremals. *Geometric and Functional Analysis*, 17(5):1343–1415, 2008.

[17] Carbery, A., Valdimarsson, S. I. The endpoint multilinear Kakeya theorem via the Borsuk–Ulam theorem. *Journal of Functional Analysis*, 264(7):1643–1663, 2013.

[18] Hickman, J., Rogers, K. M., Zhang, R. Improved bounds for the Kakeya maximal conjecture in higher dimensions. *American Journal of Mathematics*, 144(6):1511–1560, 2022.

[19] Zahl, J. On the Wolff circular maximal function. *Illinois Journal of Mathematics*, 56(4):1281–1295, 2012.

[20] Zahl, J. A new bound on Kakeya sets in $\mathbb{R}^3$. *arXiv preprint arXiv:2104.13528*, 2022.

[21] Wang, H., Zahl, J. Volume estimates for unions of convex sets, and the Kakeya set conjecture in three dimensions. *arXiv preprint arXiv:2502.17655*, 2025.

[22] Tropp, J. A. User-friendly tail bounds for sums of random matrices. *Foundations of Computational Mathematics*, 12(4):389–434, 2012.

[23] Halko, N., Martinsson, P.-G., Tropp, J. A. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.

[24] Zamir, R., Feder, M. On lattice quantization noise. *IEEE Transactions on Information Theory*, 42(4):1152–1159, 1996.

[25] Conway, J. H., Sloane, N. J. A. Fast quantizing and decoding algorithms for lattice quantizers and codes. *IEEE Transactions on Information Theory*, 28(2):227–232, 1982.

[26] Conway, J. H., Sloane, N. J. A. *Sphere Packings, Lattices and Groups*, 3rd ed. Springer, 1999.

[27] Eyuboglu, M. V., Forney, G. D. Lattice and trellis quantization with lattice- and trellis-bounded codebooks—high-rate theory for memoryless sources. *IEEE Transactions on Information Theory*, 39(1):46–59, 1993.

[28] Forney, G. D. Coset codes—part I: introduction and geometrical classification. *IEEE Transactions on Information Theory*, 34(5):1123–1151, 1988.

[29] Erez, U., Zamir, R. Achieving $1/2\log(1+\mathrm{SNR})$ on the AWGN channel with lattice encoding and decoding. *IEEE Transactions on Information Theory*, 50(10):2293–2314, 2004.

[30] Chee, J., Cai, Y., Kuleshov, V., De Sa, C. QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. *ICML*, 2024.

[31] Tseng, A., Chee, J., Sun, Q., Kuleshov, V., De Sa, C. QuIP: 2-bit quantization of large language models with guarantees. *NeurIPS*, 2023.

[32] Li, A. KakeyaLattice v1.4: the canonical implementation. Open-source release, *LLM-KV–Cache-compress*, tag `v1.4`, April 2026. Python class `kakeyaturbo_py.V14KakeyaZamirLatticeGPU`; multi-model measurement harness `benchmarks/multimodel_v14_vs_tq.py`; four per-architecture snapshot hooks in `vllm_backend/kakeya_v1_3_ppl/snapshot_hook.py`.

[33] Li, A. Randomized Kakeya Skeletons for LLM KV Cache Compression: Algorithm and Rate–Distortion Boundary. *LLM-KV–Cache-compress v1.3 paper*, April 2026. (Superseded by this paper at tag `v1.4`.)

[34] vibhavagarwal5 *et al.* TurboQuant: 2-bit KV cache compression with 4x capacity. *vLLM merged attention backend*, 2026. (Merged into vllm-project/vllm main on 2026-04-15.)

[35] vibhavagarwal5. [Attention Backend] TurboQuant: 2-bit KV cache compression with 4x capacity. *vllm-project/vllm Pull Request #38479*, 2026. https://github.com/vllm-project/vllm/pull/38479.

[36] Alistarh, D. [Attention] TurboQuant: remove redundant random signs, add prior art attribution. *vllm-project/vllm Pull Request #40194*, 2026.

[37] aditi-amd. [ROCm] Fix TurboQuant on ROCm: backend routing, flash-attn compat. *vllm-project/vllm Pull Request #39953*, 2026.

[38] mgoin. Fix TURBOQUANT backend selection in `cuda.py`. *vllm-project/vllm Pull Request #40060*, 2026.

[39] huangzhilin-hzl. [TurboQuant] Add NIAH long-context eval for TurboQuant KV cache. *vllm-project/vllm Pull Request #40122*, 2026.

[40] Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *ICML*, 2024.

[41] Shutova, E. *et al.* Cache Me If You Must: Adaptive KV Cache Quantization for LLMs. *arXiv preprint arXiv:2501.19392*, 2025.

[42] Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., Chen, B. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. *NeurIPS*, 2023.

[43] Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., Shrivastava, A. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. *NeurIPS*, 2023.

[44] Xiao, G., Tian, Y., Chen, B., Han, S., Lewis, M. Efficient streaming language models with attention sinks. *ICLR*, 2024.

[45] Nawrot, P., Lancucki, A., Chochowski, M., Tarjan, D., Ponti, E. M. Dynamic memory compression: Retrofitting LLMs for accelerated inference. *arXiv preprint arXiv:2403.09636*, 2024.

[46] Dong, H., Yang, X., Zhang, Z., Wang, Z., Chi, Y., Chen, B. Get more with LESS: Synthesizing recurrence with KV cache compression for efficient LLM inference. *ICML*, 2024.

[47] DeepSeek-AI. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437*, 2024.

[48] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., Stoica, I. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.

[49] Wolf, T. *et al.* Transformers: State-of-the-art natural language processing. *EMNLP (Demo)*, 2020.

[50] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.

A Reproducibility manifest

The full multi-model benchmark for *both codecs* is reproducible via the rigorous evaluation harness introduced with the v1.5 release:

```
cd LLM-KV--Cache-compress
pip install -e kakeyalattice
pip install -e vllm_backend
export VLLM_ENABLE_V1_MULTIPROCESSING=0 KAKEYA_SNAPSHOT_QWEN3=1

# PPL / MSE / CR (v1.4 + v1.5 + TurboQuant at matched Q / b):
python benchmarks/rigorous_eval.py \
    --model-path <HF-id> --model-name <short>_nobdry \
    --mode inforward --no-boundary \
    --q-values 4,10 --v15-q-values 4,10 --tq-b-values 3 \
    --kv-modes KV \
    --ctx-len 2048 --n-eval 64 --n-passages 32 \
    --out-dir reports/v1_5_release

# TurboQuant b=2 guardrail (requires boundary=2 to boot):
python benchmarks/rigorous_eval.py \
    --model-path <HF-id> --model-name <short>_tqb2_bdry2 \
    --mode inforward --boundary-size 2 \
    --q-values "" --v15-q-values "" --tq-b-values 2 \
    --kv-modes KV \
    --ctx-len 2048 --n-eval 64 --n-passages 32 \
    --out-dir reports/v1_5_release

# Pure codec latency (no model needed):
python benchmarks/e8_latency_benchmark.py --n-iters 500

# NIAH long-context retrieval:
python benchmarks/niah_eval.py \
    --model-path <HF-id> --model-name <short> \
    --mode inforward --boundary-size 2 --n-trials 3 \
    --ctx-lengths 4096,8192,16384 --depths 0.1,0.5,0.9 \
    --q-values 4,10 --v15-q-values 4,10 --tq-b-values 2,3 \
    --out-dir reports/v1_5_release/niah

# Frozen sha256 parity (bit-level regression gate):
python benchmarks/e8_parity_and_smoke.py
```

Replace `<HF-id>` with one of `Qwen/Qwen3-4B`, `deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B`, `google/gemma-4-E4B`, or `zai-org/GLM-4-9B-Chat` (add `--trust-remote-code` for the last). Raw per-passage JSON and full stdout logs are committed under `reports/v1_5_release/` (v1.5 data) and `reports/v1_4_release/` (v1.4 frozen data).

B Canonical naming

- **Project name**: KakeyaLattice
- **Python package**: `kakeyalattice`
- **v1.4 codec (D_4)**:
  - spoken / written: "v1.4 kakeya zamir lattice GPU"
  - with parameter: e.g. "v1.4 $Q = 152$"
  - class: `V14KakeyaZamirLatticeGPU`
  - module: `kakeyalattice.v1_4_kakeya_zamir_lattice_gpu`
  - release tag: `v1.4` (commit `6b02711`)
- **v1.5 codec (E_8)**:
  - spoken / written: "v1.5 kakeya zamir E8 GPU"
  - with parameter: e.g. "v1.5 $Q = 10$"
  - class: `V15KakeyaZamirE8GPU`
  - module: `kakeyalattice.v1_5_kakeya_zamir_e8_gpu`
  - release tag: `v1.5` (commit at the time of paper release)

C Canonical operating points

For $D = 128$ (head_dim of Qwen3-4B, DeepSeek-R1-Distill-1.5B, and GLM-4-9B-Chat):

	D_4 nested lattice variant		E_8 nested lattice variant		
q_{range}	bits	CR	bits	CR	Role
152	1088	1.88×	1104	1.86×	Near-lossless
76	960	2.13×	976	2.10×	
38	832	2.46×	848	2.41×	D_4 **balanced**
19	704	2.91×	720	2.84×	
10	576	3.56×	608	3.37×	E_8 **balanced**
5	448	4.57×	464	4.41×	
4	416	4.92×	448	4.57×	Aggressive compression
2	320	6.40×	336	6.10×	Extreme compression

For Gemma-4-E4B's heterogeneous head_dim (20 sliding-attention layers with $d_h = 256$, 4 full-attention layers with $d_h = 512$), both codecs emit per-layer bit budgets via the `LatticeCodebook` dispatcher in `kakeyalattice.lattice_codebooks`; the aggregated 128k storage figures in Table 5 use those per-layer values.

Deployment recommendation. Use E_8 **Q=**10 (CR 3.37×, $|\Delta\text{ppl}|$ typically 1.6–6.9%) as the default head-of-line operating point. Use E_8 Q=4 (CR 4.57×) when aggressive compression is non-negotiable and $|\Delta\text{ppl}|$ of 6–32% is acceptable on the target model family. Use E_8 Q=152 (CR 1.86×, $|\Delta\text{ppl}| < 1\%$) when near-lossless is required. Stay on the D_4 nested lattice variant for latency-critical paths where the 1.56–1.80× E_8 encode cost is unacceptable.